

MotifVM: A Motif-Aware Cognitive Compiler for Invariant-Checked Reasoning

Compiler Layer, Runtime Kernel, Audit Packs, and Replanning at v0.6.5

Nikit Phadke

Version 0.6.5

Abstract

LLM applications frequently collapse ingestion, reasoning, execution, planning, and trust into a single opaque loop. A model reads an artifact, emits text, calls tools, and the surrounding application often treats the output as actionable state. MotifVM proposes a different architecture: a motif-aware cognitive compiler above a small invariant-checked runtime kernel. The compiler diagnoses a task into a **MotifFrame**, computes gap and consequence-weighted risk, selects a **ReasoningPlan** from declared **PassEffect** metadata, chooses light, standard, or strict verification, and records replanning actions when terminal invariants fail. The runtime kernel remains stricter: artifacts are normalized into hash-bound evidence references and extracted facts, reasoning components propose typed state patches, and the runtime alone validates, authorizes, applies, verifies, commits, and exports terminal states. This paper describes MotifVM v0.6.5 as the first complete research release: **TaskAST** $\rightarrow$ **MotifFrame** $\rightarrow$ **ReasoningPlan** $\rightarrow$ **StatePatch** $\rightarrow$ invariants $\rightarrow$ replan or terminal state $\rightarrow$ audit pack. We present the compiler layer, runtime model, adapter contract, fact normal form, patch authorization system, invariant-driven terminal semantics, audit-pack verifier, implemented domains, evaluation gates, threat model, and limitations.

1 Introduction

Modern LLM systems are commonly described as agents, chains, or workflows. These terms emphasize orchestration: what component calls what, in what order, with what prompt. They are less precise about the semantics of state. When a model summarizes a spreadsheet, reviews a code diff, or interprets a document, where does the evidence live? Which component is allowed to mutate application state? How is a failed answer represented? Can the result be independently checked after export?

MotifVM starts from the opposite direction. It treats reasoning as a virtual-machine problem. The runtime has explicit memory, typed patches, authority references, invariants, terminal states, and portable audit packs. LLMs and tools are outside the trust boundary. They may propose outputs, but they do not directly write truth into the runtime.

The design target is an auditable cognitive VM:

Model and tool outputs do not become truth. They become proposed state transitions over explicit cognitive state.

The v0.5.4 milestone froze the core boundary around artifact adapters and an **ExtractedFact** normal form. The v0.6.5 milestone adds the missing cognitive compiler above that kernel. This is

the point at which MotifVM becomes more than an invariant-checked runtime: it can inspect the motif demands of a task, choose a pass plan, scale verification strength, and explain why a terminal failure should request evidence, run reconciliation, load authority, or commit a security failure.

2 Design Goals

MotifVM is designed around eight goals:

1. **Explicit state.** Reasoning state must be inspectable as a structured object, not implicit in chat history.
2. **Single mutation path.** All state changes must pass through `StatePatch`.
3. **Adapter isolation.** Artifact ingestion must be separated from domain reasoning.
4. **Evidence stability.** Every evidence record must be hash-bound and locator-stable.
5. **Fact/claim separation.** Extracted facts are weaker than claims and must not imply terminal conclusions.
6. **Patch authorization.** LLMs, passes, tools, reconciliation logic, and runtime code must have different permissions.
7. **Invariant terminal semantics.** Success and failure must be committed terminal states with typed failure classes.
8. **Portable verification.** Audit packs must be checkable without rerunning the original task.
9. **Motif-aware planning.** Pass selection and verification strength must be derived from explicit motif gaps and risk, not from a fixed workflow.

3 System Overview

The core MotifVM transition law is:

$$\text{CognitiveState} + \text{StatePatch} \rightarrow \text{validate} \rightarrow \text{authorize} \rightarrow \text{apply} \rightarrow \text{verify} \rightarrow \text{commit}$$

The artifact ingestion law is:

$$\text{Artifact} \rightarrow \text{ArtifactAdapter} \rightarrow \text{EvidenceRef}^* \rightarrow \text{ExtractedFact}^*$$

Domain passes then consume extracted facts and propose claims through patches:

$$\text{ExtractedFact}^* \rightarrow \text{DomainPass} \rightarrow \text{StatePatch} \rightarrow \text{Claim}^*$$

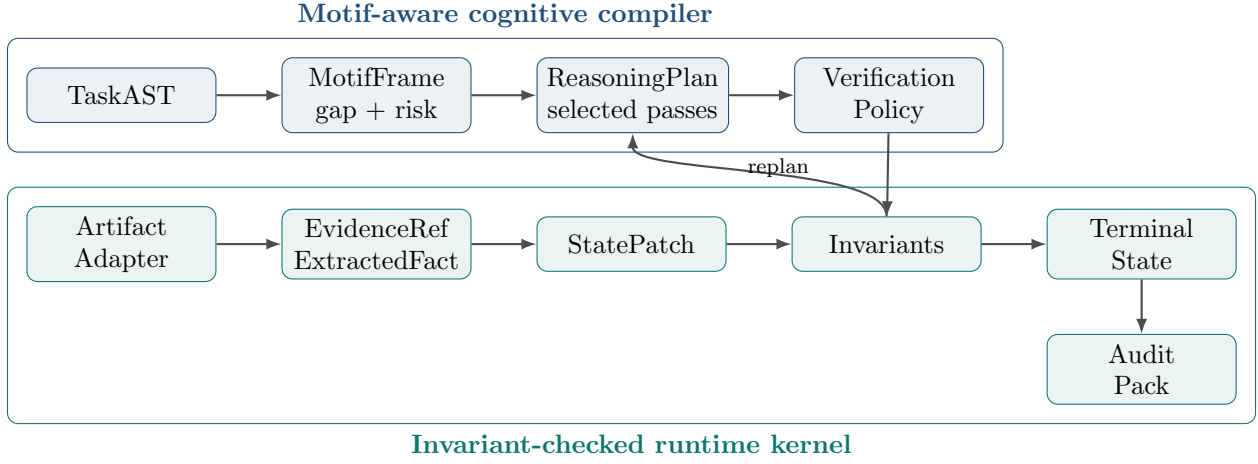
The complete path is therefore:

$$\begin{aligned} &\text{ArtifactAdapter} \rightarrow \text{EvidenceRef} \rightarrow \text{ExtractedFact} \rightarrow \text{StatePatch} \\ &\quad \rightarrow \text{Invariant} \rightarrow \text{TerminalState} \rightarrow \text{AuditPack} \rightarrow \text{PackVerifier} \end{aligned}$$

At v0.6.5, the complete compiler-over-runtime law is:

$$\begin{aligned} &\text{TaskAST} \rightarrow \text{MotifFrame} \rightarrow \text{ReasoningPlan} \rightarrow \text{VerificationPolicy} \\ &\quad \rightarrow \text{StatePatch}^* \rightarrow \text{Invariant}^* \rightarrow \text{Replan or TerminalState} \rightarrow \text{AuditPack} \end{aligned}$$

3.1 Compiler over Runtime Architecture



4 Kernel Primitives

Primitive	Role
<code>CognitiveState</code>	Explicit runtime memory containing task AST, graph, artifacts, invariants, pass history, patch timeline, and terminal metadata.
<code>StatePatch</code>	The only mutation vehicle for cognitive state.
<code>PatchAuthorization</code>	Role-based permission boundary for patches.
<code>ArtifactAdapter</code>	Device-driver-like ingestion component for external artifacts.
<code>EvidenceRef</code>	Hash-bound pointer to a stable region of an input artifact.
<code>ExtractedFact</code>	Normalized fact extracted from evidence before domain claims are made.
<code>Claim</code>	Runtime assertion produced by a domain pass and traceable to evidence.
<code>Invariant</code>	Predicate over state that determines correctness, warnings, or terminal failure.
<code>AuthorityRef</code>	Source-bound rule reference with section metadata and hashes.
<code>TerminalState</code>	Committed success or structured committed failure.
<code>AuditPack</code>	Portable bundle containing state, graph, facts, lineage, invariants, inputs, and reports.
<code>PackVerifier</code>	Independent consistency checker for exported audit packs.
<code>PatchTimeline</code>	Ordered record of authorized and rejected patch proposals.
<code>MotifFrame</code>	Compiler record of required support, current support, gap, consequence-weighted risk, and selected policies.
<code>ReasoningPlan</code>	Compiler output containing selected passes, model/tool/checkpoint/reconciliation policies, verification policy, expected gap reduction, and rationale.
<code>PassEffect</code>	Metadata describing each pass’s motif deltas, cost profile, risk profile, and failure modes.

5 Motif-Aware Cognitive Compiler

The frozen kernel answers the question: what counts as a valid state transition? The compiler answers a different question: which transitions should this task attempt, and under what verification policy?

5.1 MotifFrame

Every run begins with a `TaskAST` and a motif diagnosis. The compiler constructs a `MotifFrame`:

$$\text{MotifFrame.risk}[m] = \max(0, \text{required}[m] - \text{supported}[m]) \times \text{consequence}[m]$$

The frame records required support, current support, gap, risk, selected compiler policies, creator, and timestamp. Consequence is domain-sensitive: a missing invariant in a regulated CRAR audit or security review is more consequential than missing invariant strength in a low-risk summary.

5.2 PassEffect Registry

Each runtime pass declares a `PassEffect` entry:

```
{
  "strengthens": {"invariant": 0.35, "authority": 0.25},
  "cost": {"latency": 0.12, "tokens": 0.0, "toolRisk": 0.0},
  "risk": {"sourceMutation": 0.0, "llmReliance": 0.0},
  "failureModes": ["authority_missing", "invariant_failed"],
  "runtimeInternal": false
}
```

The compiler refuses to build a plan when a non-internal pass lacks effect metadata. This makes pass selection inspectable: the planner is not an implicit ordering hidden in code; it is a decision over declared effects.

5.3 ReasoningPlan

The planner scores candidate passes by expected motif-gap reduction minus cost:

$$\text{score}(p) = \sum_m \text{gap}[m] \cdot \text{effect}_p[m] - \text{costPenalty}(p)$$

It then includes dependencies and emits a topologically valid **ReasoningPlan**. High-authority or high-invariant tasks select authority and structural verification passes. Low-risk summary tasks select a cheaper plan and light verification. The plan records selected passes, model policy, tool policy, checkpoint policy, reconciliation policy, verification policy, expected gap reduction, and rationale.

5.4 Adaptive Verification

Verification is selected from motif risk:

Policy	Checks
light	Schema and terminal checks for low-risk or scarcity-dominant tasks.
standard	Schema, terminal, lineage, authority, and domain invariant checks.
strict	Standard checks plus contradiction scan, confidence propagation, and pack-verifier expectations.

5.5 Replanning

When fatal invariants fail, MotifVM classifies the failure and appends a compiler replan event. The implemented replan actions include request more evidence, load authority sources, run reconciliation, commit failure with a clean-input request, and commit security failure. Replans are exported in `reasoning_plans.json` and recorded in the patch timeline as `motif_compiler_replan`.

6 CognitiveState

CognitiveState is the VM memory image. In v0.5.4 it contains:

- task AST and domain metadata,
- authority references,

- input manifest,
- motif vector state,
- reasoning graph,
- artifacts,
- decisions,
- invariant results,
- pass history,
- patch timeline,
- execution log,
- terminal status, reason, and failure class.

The state object is deliberately verbose. The goal is not minimal storage; the goal is inspectability. A committed state should explain not only what answer was produced, but which inputs, evidence, facts, authorities, invariants, and patches produced it.

7 Artifact Adapters

An artifact adapter is the boundary between unstructured or semi-structured external artifacts and the VM. It is analogous to a device driver in an operating system. A disk driver does not change the kernel's process model; it gives the kernel a disciplined way to interact with a disk. Similarly, a CSV adapter or PDF adapter should not change MotifVM's reasoning semantics; it should produce stable evidence and normalized facts.

7.1 Adapter Contract

Each adapter has:

- an adapter identifier,
- a version,
- supported media or path types,
- a deterministic `adapt(input_ref, path)` function.

Each adapter emits an `AdapterResult`:

- adapter id and version,
- input id,
- path,
- content hash,

- zero or more resolved input artifacts,
- `EvidenceRef[]` records,
- `ExtractedFact[]` records,
- adapter metadata.

Adapters may not mutate `CognitiveState`. They may not emit terminal claims. They may not mark invariants as passed. They are perception components, not reasoning components.

7.2 Built-In Adapters

v0.6.5 includes three built-in adapters:

Adapter	Input	Facts emitted
<code>adapter:csv:v0.5</code>	CSV files	<code>dccb_component</code> facts
<code>adapter:git_diff:v0.5</code>	Patch files	<code>code_added_line</code> facts
<code>adapter:repo:v0.5</code>	Repository directories	<code>code_added_line</code> , <code>repo_changed_file</code> facts

8 EvidenceRef

`EvidenceRef` is the stable evidence pointer. Its purpose is to preserve a durable relation between an extracted item and the artifact region it came from.

```
EvidenceRef {
  id
  inputId
  inputHash
  locatorType
  locator
  excerpt
  metadata
}
```

Examples of locator types include rows, lines, cells, byte ranges, JSON paths, and file regions. The input hash matters because a line number without a content hash is not evidence; it is only a coordinate.

9 ExtractedFact Normal Form

`ExtractedFact` is the normal form between raw evidence and domain reasoning.

```
ExtractedFact {
  id
  kind
  value
  evidenceRefId
}
```

```

confidence
extractedBy
domainHints
metadata
}

```

Facts are not claims. A fact says what was extracted from an artifact. A claim says what a domain pass concludes from facts under authority and invariants. This distinction prevents domain assumptions from leaking into adapters.

9.1 Examples

Artifact fragment	ExtractedFact	Possible claim
<code>tier1,120</code>	<code>dccb_component</code>	Computed CRAR uses Tier I capital.
<code>return True</code>	<code>code_added_line</code>	Authorization bypass detected.
<code>auth.py changed</code>	<code>repo_changed_file</code>	Reviewed file is hash-bound.

10 Facts to Claims

Domain passes consume extracted facts and propose claims through `StatePatch`. In the DCCB domain, component facts become CRAR computation claims. In the code-review domain, added-line facts become security findings and risk claims.

This stage is where domain knowledge enters. The adapter does not know that a CRAR threshold matters. The DCCB pass does. The adapter does not know that `return True` is unsafe in an authorization function. The code-review pass does. The runtime then checks whether the proposed claims satisfy invariants.

11 StatePatch Semantics

`StatePatch` is the syscall-like mutation interface for MotifVM. A patch may contain:

- nodes to add,
- nodes to update,
- edges to add,
- artifacts to add,
- decisions to add,
- task updates,
- status update,
- motif support deltas.

Before a patch is applied, the runtime checks:

1. schema validity,

2. graph endpoint validity,
3. duplicate node constraints,
4. motif delta validity,
5. role authorization.

12 Patch Authorization

Patch authorization is a kernel-level concern because every actor must obey it. v0.6.5 uses role policies:

Role	Allowed	Blocked examples
Domain pass	Add domain-scoped claims, outputs, edges, and artifacts.	Cross-domain claim injection.
LLM	Add bounded narrative artifacts.	Terminal status, invariant results, input manifest, authority refs, direct reconciliation patches.
Runtime	Commit state and write runtime metadata.	Unverified terminal facts.
Reconciliation	Add non-mutating correction proposals.	Source mutation without confirmation.

The important point is that LLM output can remain useful without being trusted as state. A model may summarize the final result, but it may not silently change the terminal status or mark failed invariants as passed.

13 Reasoning Graph

The reasoning graph connects evidence, claims, outputs, assumptions, decisions, and errors. Graph edges encode relations such as **supports**, **produces**, **contradicts**, and **blocks**. This graph is not decorative; invariants inspect it. Audit packs export it in JSON, DOT, and Mermaid forms.

The graph lets MotifVM answer questions such as:

- Which evidence supports this claim?
- Which authority references govern this invariant?
- Which claim produced this output?
- Which contradiction caused reconciliation?
- Which patch introduced this artifact?

14 Invariants and Terminal States

Invariants define whether a state can be committed as success or must be committed as structured failure. MotifVM distinguishes failure classes rather than collapsing all failures into generic exceptions.

Failure class	Meaning
<code>input_invalid</code>	Malformed or invalid artifact values.
<code>computation_blocked</code>	Required evidence is missing or insufficient.
<code>reconciliation_required</code>	Evidence conflicts and a non-mutating correction is proposed.
<code>security_risk_detected</code>	Code-review invariants detect unsafe behavior.
<code>llm_patch_rejected</code>	A model proposed a forbidden transition.
<code>authority_missing</code>	Required authority references are absent.
<code>lineage_missing</code>	Terminal outputs lack traceable evidence.

This makes failure part of the VM output. A failed run still preserves evidence, graph state, invariant failures, reconciliation artifacts, and audit metadata.

15 Authority References

`AuthorityRef` records source-bound rule references. v0.4.5 introduced section-level hardening:

- source hash,
- section id,
- quoted rule excerpt,
- effective date,
- retrieval timestamp,
- section hash.

This is intentionally snapshot-based. MotifVM does not browse live regulatory sources in the current kernel; it verifies checked-in authority snapshots.

16 Audit Packs

An audit pack is a portable evidence bundle for a committed state. v0.6.5 audit packs include:

- `state.json`,
- `report.md`,
- `motif_frame.json`,
- `reasoning_plan.json`,
- `reasoning_plans.json`,

- `graph.json`, `graph.dot`, `graph.mmd`,
- `lineage.json`,
- `invariants.json`,
- `inputs_manifest.json`,
- `extracted_facts.json`,
- `llm_calls.json`,
- `reconciliation_patch.json`,
- `patch_timeline.json`,
- `patch_timeline.md`,
- artifact JSON files.

17 Pack Verification

The pack verifier checks internal consistency without rerunning the task. It verifies:

- required files,
- input file and directory hashes,
- authority source hashes,
- authority section hashes,
- graph endpoints,
- lineage references,
- reconciliation policy,
- report terminal status consistency,
- patch timeline shape,
- compiler frame and reasoning plan shape,
- adapter output conformance.

This changes the trust model from “trust my answer” to “inspect this evidence bundle and verify its internal consistency.”

18 Implementation

The current implementation is a dependency-light Python package. The runtime exposes commands for running tasks, rendering reports, exporting audit packs, comparing runs, running evaluations, running adversarial suites, verifying packs, and checking adapter conformance.

```
make test
make compiler-eval
make adapter-conformance
make demo
make eval
make adversarial
make adversarial-100
python3 -m motifvm verify-pack <audit-pack-dir>
```

The implementation includes two domain profiles:

- **DCCB CRAR audit:** computes CRAR, checks threshold compliance, detects reported versus computed conflicts, and proposes non-mutating reconciliation patches.
- **Code review:** reviews patch and repository inputs for security-sensitive findings such as authorization bypass, secret literals, shell execution, eval/exec, disabled TLS verification, SQL interpolation, and unsafe deserialization.

19 Evaluation

MotifVM’s release gates are intentionally modest but concrete. The goal is to verify runtime boundaries, not to claim universal domain performance.

Version	Core proof	Gate
0.1.8	Kernel and two-domain audit packs	Canonical demo runs
0.2.5	Provider boundary and evaluation	7-case eval harness
0.3.5	Adversarial reliability	20-case adversarial suite
0.4.5	Scale and integration	Repository review + adversarial-100
0.5.4	Adapter boundary freeze	Adapter conformance + extracted facts
0.6.5	Cognitive compiler	MotifFrame + ReasoningPlan + compiler eval

The v0.6.5 release gate observed:

- 21 unit tests passing,
- compiler evaluation passing,
- adapter conformance passing for CSV, git diff, and repository adapters,
- demo suite passing,
- evaluation suite passing,

- 20-case adversarial suite passing,
- 100-case adversarial suite passing,
- representative audit pack verification passing,
- PDF generation passing.

19.1 Combined Evaluation Table

Evaluation	Metric	Observed result
Runtime eval	terminal status accuracy	1.0 across 7 cases
Runtime eval	failed invariant accuracy	1.0 across 7 cases
Runtime eval	reconciliation accuracy	1.0 across 7 cases
Runtime eval	lineage, authority, input-hash completeness	1.0 each
Runtime eval	schema validation failures	0
Adversarial eval	20-case status, invariant, failure-class correctness	20/20
Adversarial-100	terminal accuracy	1.0000
Adversarial-100	failed invariant accuracy	1.0000
Adversarial-100	failure-class accuracy	1.0000
Adversarial-100	pack verifier pass rate	1.0000
Adversarial-100	false positive / false negative rate	0.0000 / 0.0000
Compiler eval	motif diagnosis accuracy	4/4 qualitative task-risk labels
Compiler eval	plan selection accuracy	4/4
Compiler eval	verification policy accuracy	4/4
Compiler eval	missed required pass rate	0/4
Compiler eval	unnecessary pass rate	0/4

20 Example Compiler Trace

This trace uses the DCCB reported-CRAR mismatch fixture. It demonstrates the full path from task to compiler frame, plan, patches, invariant failure, replan, and terminal state.

1. **Task.** The user asks for CRAR verification over the mismatch fixture with domain `dccb-audit`.
2. **MotifFrame.** The compiler diagnoses high addressing, state, invariant, authority, reconciliation, and terminal-state demand. Because this is a regulated audit domain, invariant and authority gaps receive high consequence weights. The selected policies include strict verification and authority requirements.
3. **ReasoningPlan.** The compiler selects the dependency-safe pass sequence: normalize intent, resolve inputs, extract constraints, build evidence graph, decompose task, assign tools, schedule execution, execute subtasks, verify structural, and synthesize output. The verification policy is `strict`.

4. **StatePatches.** Runtime passes emit authorized patches. The input adapter produces hash-bound CSV evidence and `dccb_component` facts. Domain execution computes CRAR from Tier I, Tier II, and RWA facts and adds claim/output nodes.
5. **Invariant failure.** The runtime checks `DCCB_003_REPORTED_CRAR_MATCH`. The reported value is 19.00 percent while the computed value is 16.00 percent, so the invariant fails with severity `error`.
6. **Reconciliation patch.** Reconciliation logic proposes a non-mutating correction artifact and adds a corrected-output node blocked by auditor confirmation. The original source artifact is not modified.
7. **Compiler replan.** The failure classifier emits `reconciliation_required`. The compiler appends a `motif_compiler_replan` patch-timeline event with action `run_reconciliation` and increases reconciliation/feedback risk in the motif frame.
8. **Terminal state.** The state commits as failed with failure class `reconciliation_required`. The audit pack exports state, graph, invariants, lineage, extracted facts, patch timeline, motif frame, reasoning plan, and replan history.

21 Threat Model

MotifVM protects the following assets:

- cognitive state,
- motif frames and reasoning plans,
- input manifests and hashes,
- authority references and hashes,
- extracted facts and evidence refs,
- invariant results,
- terminal state,
- audit packs.

Threats considered include malformed input, adversarial numeric fields, unsafe code diffs, LLM attempts to modify terminal facts, unauthorized source mutation, missing lineage, over- or under-verification by the compiler, and audit-pack tampering.

Controls include schema validation, patch authorization, pass-effect metadata, motif-risk-based verification policy, input and authority hashing, domain and universal invariants, failure taxonomy, patch timeline export, and independent pack verification.

22 Limitations

MotifVM does not prove that every domain rule is complete, every source authority is legally current, every planner choice is globally optimal, or every possible security flaw is detected. It does not replace static analyzers, regulatory counsel, formal methods, or human review. It currently uses checked-in authority snapshots rather than live source retrieval. Repository review is still patch-oriented rather than a full program analysis engine.

The verifier checks internal consistency of the exported audit pack. It does not certify that the domain is complete or that the output is legally or financially sufficient.

23 Future Work

With the adapter boundary frozen and the compiler layer implemented, future work should primarily expand surfaces and evaluations rather than mutate the kernel:

- XLSX, PDF, JSON, log, API, database, and email adapters,
- richer domain profiles,
- larger labeled evaluations,
- more rigorous compiler-planning benchmarks,
- stronger static-analysis integration for code review,
- source snapshotting for external authorities,
- interactive audit-pack inspection UI,
- deployment patterns for CI and regulated workflows.

24 Conclusion

MotifVM is a motif-aware cognitive compiler over an invariant-checked reasoning runtime. Its core contribution is the disciplined treatment of model and tool output as proposed state transitions rather than truth. Tasks become MotifFrames and ReasoningPlans. Artifacts enter through adapters. Adapters emit evidence and facts. Domain passes propose claims. StatePatches are validated and authorized. Invariants define terminal states and replan actions. Audit packs preserve the trace. The verifier checks the exported bundle.

This architecture separates planning, perception, reasoning, trust, mutation, and portability. The kernel remains small; the compiler decides how to approach the task; adapters and domains decide how the outside world enters. That separation is the system.